

Buen diseño - Cohesión y Acoplamiento

Profesor: Teófilo Chambilla

Objetivo: La presente tarea tiene como objetivo que el estudiante diseñe e implemente una solución de software aplicando principios de buen diseño, arquitectura de software y calidad de código. La solución deberá evidenciar modularidad, abstracción, bajo acoplamiento y alta cohesión, considerando un escenario basado en mensajería y procesamiento de eventos.

- Diseñar una solución de software que soporte los atributos de **modularidad, abstracción, bajo acoplamiento y alta cohesión**.
- Implementar la solución utilizando un patrón arquitectónico adecuado, como **Arquitectura Hexagonal, Clean Architecture, Microservicios o Arquitectura Orientada a Eventos**.
- Modelar el sistema mediante un **Diagrama de Casos de Uso**, identificando actores, funcionalidades principales y relaciones relevantes.
- Incorporar un mecanismo de mensajería utilizando **RabbitMQ, Apache Kafka o ActiveMQ**, según el enfoque técnico elegido por el equipo.
- Aplicar buenas prácticas de calidad de software, incluyendo pruebas automatizadas, análisis estático de código y control de duplicidad.

Introducción Ir a un restaurante suele representar un gasto mayor que cocinar en casa; sin embargo, los programas de recompensas y fidelización permiten que los clientes obtengan beneficios por sus consumos. Estos programas ofrecen acumulación de puntos, reembolsos o beneficios especiales cada vez que un cliente consume en restaurantes afiliados.

Por ejemplo, Jesús desea ahorrar dinero para la educación de sus hijos. Cada vez que realiza una cena en un restaurante participante, una parte del consumo es transformada en puntos o recompensas que son abonadas a su cuenta personal.

Actualmente, debido a la necesidad de procesar grandes volúmenes de transacciones en tiempo real, las empresas utilizan arquitecturas orientadas a eventos y plataformas de mensajería como **RabbitMQ, Apache Kafka o ActiveMQ**, permitiendo desacoplar los sistemas y mejorar la escalabilidad, disponibilidad y resiliencia de las aplicaciones.

Implemente el proceso indicado en la Figura 1 considerando una arquitectura basada en mensajería y eventos:

- El restaurante registra la información de la cena realizada por el cliente.

- El sistema del restaurante procesa internamente la transacción y publica un mensaje en un **Broker de Mensajería (RabbitMQ, Apache Kafka o ActiveMQ)** con la siguiente información:
 - Monto consumido.
 - Número de tarjeta del cliente.
 - Código del restaurante afiliado.
 - Fecha y hora de la transacción.
- El Broker de Mensajería se encarga de la administración de colas, tópicos o eventos, garantizando la entrega de mensajes entre productores y consumidores.
- Un microservicio consumidor correspondiente al sistema de recompensas recibe el mensaje y calcula automáticamente los puntos, cashback o beneficios asociados al cliente.
- El sistema actualiza la cuenta de recompensas del cliente.
- Opcionalmente, el sistema puede publicar un nuevo evento para el envío de una notificación por correo electrónico, SMS o aplicación móvil indicando que la recompensa fue procesada exitosamente.

Consideraciones Técnicas El diseño debe considerar principios de:

- Alta cohesión.
- Bajo acoplamiento.
- Modularidad.
- Escalabilidad.
- Arquitectura orientada a eventos.

Asimismo, se recomienda aplicar algún patrón arquitectónico como:

- Arquitectura Hexagonal.
- Microservicios.
- Event-Driven Architecture (EDA).
- Clean Architecture.

Entregable El proyecto deberá ser analizado mediante la plataforma **SonarCloud** con el objetivo de evaluar y mejorar la calidad del software desarrollado. El equipo deberá evidenciar buenas prácticas de ingeniería de software relacionadas con mantenibilidad, seguridad, confiabilidad y pruebas automatizadas.

El proyecto deberá alcanzar métricas vistas en clase en los siguientes atributos de calidad:

- *Reliability* (Confiabilidad).
- *Security* (Seguridad).
- *Maintainability* (Mantenibilidad).
- *Duplications* (Duplicación de código).

Asimismo, el sistema deberá alcanzar una cobertura mínima de pruebas (*Test Coverage*) del **85%**.

Para evidenciar el cumplimiento de estas actividades, se deberá subir a **Canvas** lo siguiente:

- Enlace público del análisis realizado en **SonarCloud** (Coordinar con los ACL del curso).
- Enlace del repositorio del proyecto en **GitHub**.
- Evidencia de ejecución de pruebas automatizadas.
- Documento breve describiendo la arquitectura implementada y el patrón arquitectónico utilizado.

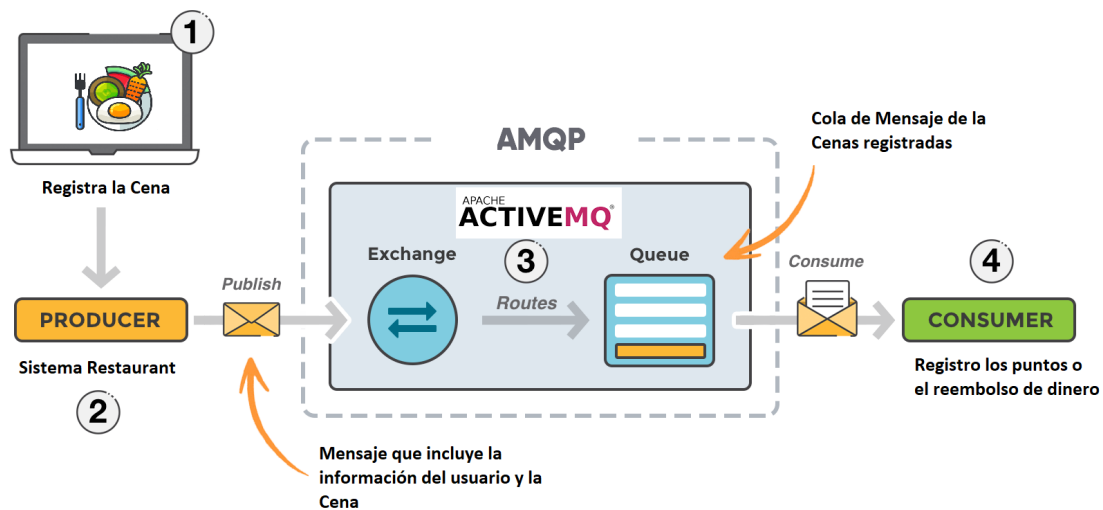


Figure 1: Proceso del programa de recompensas.